

# Step-by-Step Guide: Classes, Objects, and value assigning in C++

## 1. Understanding Classes and Objects

- **Class:** A user-defined blueprint that groups **data (attributes)** and **functions (methods)**.  
Example: A Person class with name and age as attributes.
- **Object:** An **instance** of a class.  
Example: Person person1; creates an object named person1.

## 2. Creating a Class Without Constructors

- **Define attributes as private** to enforce encapsulation (data hiding).
- **Use public methods** to set/get values.

```
#include <iostream>
using namespace std;

class Person {
private:
    string name; // Private attribute
    int age;      // Private attribute

public:
    // Setter for name
    void setName(string n) {
        name = n;
    }

    // Setter for age (with validation)
    void setAge(int a) {
        if (a > 0) age = a;
        else cout << "Invalid age!" << endl;
    }

    // Getter for name
    string getName() {
        return name;
    }

    // Getter for age
    int getAge() {
        return age;
    }
}
```

## 3. Creating an Object

- Use the default constructor (automatically generated by C++).

---

```
cpp
```

```
Person person1; // Object created without initial values
```

---

#### 4. Setting Values Using Setters

- Call setter methods to assign values to private attributes.

```
cpp
```

```
person1.setName("Alice"); // Set name
person1.setAge(25);        // Set age (valid)
person1.setAge(-5);        // Output: "Invalid age!" (age not updated)
```

---

#### 5. Getting Values Using Getters

- Use getter methods to retrieve values.

```
cpp
```

```
cout << "Name: " << person1.getName() << endl; // Output: "Alice"
cout << "Age: " << person1.getAge() << endl;    // Output: 25
```

#### 6. Full Code Example

```

#include <iostream>
using namespace std;

class Person {
private:
    string name;
    int age;

public:
    // Setters
    void setName(string n) { name = n; }
    void setAge(int a) {
        if (a > 0) age = a;
        else cout << "Invalid age!" << endl;
    }

    // Getters
    string getName() { return name; }
    int getAge() { return age; }
};

int main() {
    Person person1;
    person1.setName("Bob");
    person1.setAge(30);

    cout << "Name: " << person1.getName() << endl; // Output: Bob
    cout << "Age: " << person1.getAge() << endl;    // Output: 30
}

```

```

    person1.setAge(-5); // Output: "Invalid age!"
    return 0;
}

```

## C++ Access Modes (Access Specifiers)

Access modes define the **visibility** and **accessibility** of class members (attributes/methods) within a program. They ensure **encapsulation** (data hiding) and control how data is modified or accessed.

### 1. Types of Access Modes

There are **3 access specifiers** in C++:

| Access Mode | Description                                                              |
|-------------|--------------------------------------------------------------------------|
| public      | Members are accessible anywhere (inside/outside the class).              |
| private     | Members are only accessible within the class (default for classes).      |
| protected   | Members are accessible within the class and its derived (child) classes. |

## 2. Default Access Modes

- **For class:** Default access mode is **private**.
- **For struct:** Default access mode is **public**.

## 3. public Access Mode

- Members are accessible **everywhere** (inside the class, outside the class, and in derived classes).

Example:

```
#include <iostream>
using namespace std;

class Person {
public: // Public access
    string name;
    int age;
};

int main() {
    Person p1;
    p1.name = "Alice"; // Direct access allowed (public)
    p1.age = 25;
    cout << p1.name << " is " << p1.age << " years old." << endl;
    return 0;
}
```

## 4. private Access Mode

- Members are **only accessible within the class**.
- Use **getters/setters** to modify/access private data safely.

Example:

```
#include <iostream>
using namespace std;

class BankAccount {
private: // Private access
    double balance;

public:
    // Setter (public method to modify private data)
    void setBalance(double amount) {
        if (amount >= 0) balance = amount;
        else cout << "Invalid balance!" << endl;
    }

    // Getter (public method to access private data)
    double getBalance() {
        return balance;
    }
};

int main() {
    BankAccount acc;
    acc.setBalance(1000); // Valid
    cout << "Balance: $" << acc.getBalance() << endl;

    // acc.balance = -500; // Error: 'balance' is private
    return 0;
}
```

## . protected Access Mode

- Members are accessible within the class **and its derived (child) classes**.
- Used in **inheritance**.

```

#include <iostream>
using namespace std;

class Animal {
protected: // Protected access
    string name;
};

class Dog : public Animal { // Dog inherits from Animal
public:
    void setName(string n) {
        name = n; // Access allowed (protected in parent class)
    }
    void bark() {
        cout << name << " says Woof!" << endl;
    }
};

int main() {
    Dog d;
    d.setName("Buddy");
    d.bark(); // Output: Buddy says Woof!

    // d.name = "Max"; // Error: 'name' is protected
    return 0;
}

```

## 6. Key Rules

- **Encapsulation:** Use private to hide data and control access via public methods.
- **Inheritance:** Use protected to allow child classes to access parent members.
- **Direct Access:** Avoid public for sensitive data to prevent misuse.

## 7. Best Practices

1. **Data Hiding:** Declare attributes as private and use public getters/setters.
2. **Validation:** Add checks in setters (e.g., if (age > 0)).
3. **Readability:** Use const for getters (e.g., int getAge() const { ... }).

## 8. Common Mistakes

- Trying to access private members directly:

```
class Student {
    private:
        int roll;
};

Student s;
s.roll = 101; // Error: 'roll' is private
```

- **Forgetting to declare access modes explicitly (default is private for classes).**

### Summary Table

| Scenario                     | public | private | protected |
|------------------------------|--------|---------|-----------|
| Access inside the same class | ✓      | ✓       | ✓         |
| Access outside the class     | ✓      | ✗       | ✗         |
| Access in derived classes    | ✓      | ✗       | ✓         |

### Final Notes

- Use **public** for methods that need to be called externally.
- Use **private** for data that should not be directly modified.
- Use **protected** for data/methods shared with child classes.

### \_\_ 1: Student Class

```

#include <iostream>
using namespace std;

class Student {
private:
    string name;
    int roll;
public:
    // Setters
    void setName(string n) { name = n; }
    void setRoll(int r) { roll = r; }
    // Getters
    string getName() { return name; }
    int getRoll() { return roll; }
};

int main() {
    Student s;
    s.setName("Alice");
    s.setRoll(101);
    cout << "Student: " << s.getName() << ", Roll: " << s.getRoll() << endl;
    return 0;
}

```

## Example 2: Car Class

```

#include <iostream>
using namespace std;

class Car {
private:
    string model;
    int speed;
public:
    void setModel(string m) { model = m; }
    void setSpeed(int s) { speed = s; }
    string getModel() { return model; }
    int getSpeed() { return speed; }
};

int main() {
    Car c;
    c.setModel("Tesla Model S");
    c.setSpeed(120);
    cout << "Model: " << c.getModel() << ", Speed: " << c.getSpeed() << " km/h" << endl;
    return 0;
}

```

## Example 3: Book Class



```

#include <iostream>
using namespace std;

class Book {
private:
    string title;
    int pages;
public:
    void setTitle(string t) { title = t; }
    void setPages(int p) { pages = p; }
    string getTitle() { return title; }
    int getPages() { return pages; }
};

int main() {
    Book b;
    b.setTitle("C++ Basics");
    b.setPages(300);
    cout << "Book: " << b.getTitle() << ", Pages: " << b.getPages() << endl;
    return 0;
}

```

## Example 4: BankAccount Class

```

#include <iostream>
using namespace std;

class BankAccount {
private:
    string accountNumber;
    double balance;
public:
    void setAccountNumber(string acc) { accountNumber = acc; }
    void setBalance(double bal) {
        if (bal >= 0) balance = bal;
        else cout << "Invalid balance!" << endl;
    }
    string getAccountNumber() { return accountNumber; }
    double getBalance() { return balance; }
};

int main() {
    BankAccount acc;
    acc.setAccountNumber("ACC123");
    acc.setBalance(1000.50);
    cout << "Account: " << acc.getAccountNumber() << ", Balance: $" << acc.getBalance() << endl;
    return 0;
}

```

## Example 5: Mobile Class

```
#include <iostream>
using namespace std;

class Mobile {
private:
    string brand;
    double price;
public:
    void setBrand(string b) { brand = b; }
    void setPrice(double p) {
        if (p > 0) price = p;
        else cout << "Price must be positive!" << endl;
    }
    string getBrand() { return brand; }
    double getPrice() { return price; }
};

int main() {
    Mobile m;
    m.setBrand("Samsung");
    m.setPrice(499.99);
    cout << "Brand: " << m.getBrand() << ", Price: $" << m.getPrice() << endl;
    return 0;
}
```

## Example 6: Employee Class

```

#include <iostream>
using namespace std;

class Employee {
private:
    string name;
    double salary;
public:
    void setName(string n) { name = n; }
    void setSalary(double s) {
        if (s >= 0) salary = s;
        else cout << "Salary can't be negative!" << endl;
    }
    string getName() { return name; }
    double getSalary() { return salary; }
};

int main() {
    Employee emp;
    emp.setName("John Doe");
    emp.setSalary(50000);
    cout << "Employee: " << emp.getName() << ", Salary: $" << emp.getSalary() << endl;
    return 0;
}

```

## Example 7: Rectangle Class

```

#include <iostream>
using namespace std;

class Rectangle {
private:
    int length;
    int width;
public:
    void setLength(int l) { length = l; }
    void setWidth(int w) { width = w; }
    int getArea() { return length * width; }
};

int main() {
    Rectangle rect;
    rect.setLength(5);
    rect.setWidth(4);
    cout << "Area: " << rect.getArea() << endl; // Output: 20
    return 0;
}

```

## Example 8: Circle Class

```

#include <iostream>
using namespace std;

class Circle {
private:
    double radius;
public:
    void setRadius(double r) { radius = r; }
    double getArea() { return 3.1416 * radius * radius; }
};

int main() {
    Circle c;
    c.setRadius(7.0);
    cout << "Area: " << c.getArea() << endl; // Output: ~153.938
    return 0;
}

```

## Example 9: Animal Class

```

#include <iostream>
using namespace std;

class Animal {
private:
    string name;
    int age;
public:
    void setName(string n) { name = n; }
    void setAge(int a) { age = a; }
    string getName() { return name; }
    int getAge() { return age; }
};

int main() {
    Animal dog;
    dog.setName("Buddy");
    dog.setAge(3);
    cout << "Animal: " << dog.getName() << ", Age: " << dog.getAge() << endl;
    return 0;
}

```

## Example 10: Laptop Class

```

#include <iostream>
using namespace std;

class Laptop {
private:
    string model;
    int ram; // in GB
public:
    void setModel(string m) { model = m; }
    void setRAM(int r) { ram = r; }
    string getModel() { return model; }
    int getRAM() { return ram; }
};

int main() {
    Laptop lp;
    lp.setModel("Dell XPS");
    lp.setRAM(16);
    cout << "Model: " << lp.getModel() << ", RAM: " << lp.getRAM() << "GB" << endl;
    return 0;
}

```

## Key Notes

1. **Setters:** Methods to assign values to private attributes.
2. **Getters:** Methods to retrieve values from private attributes.
3. **Encapsulation:** Data is protected and accessed via public methods.
4. **Validation:** Added in setters (e.g., positive balance/price).